MÉTODOS DE ENSAMBLE

APRENDIZAJE DE MAQUINA - CEIA - FIUBA

Dr. Ing. Facundo Adrián Lucianna

Esp. Lic. María Carina Roldán

REPASO CLASE ANTERIOR

- Arboles de decisión
  - Arboles de regresión
  - Arboles de clasificación

ARBOLES DE DECISIÓN

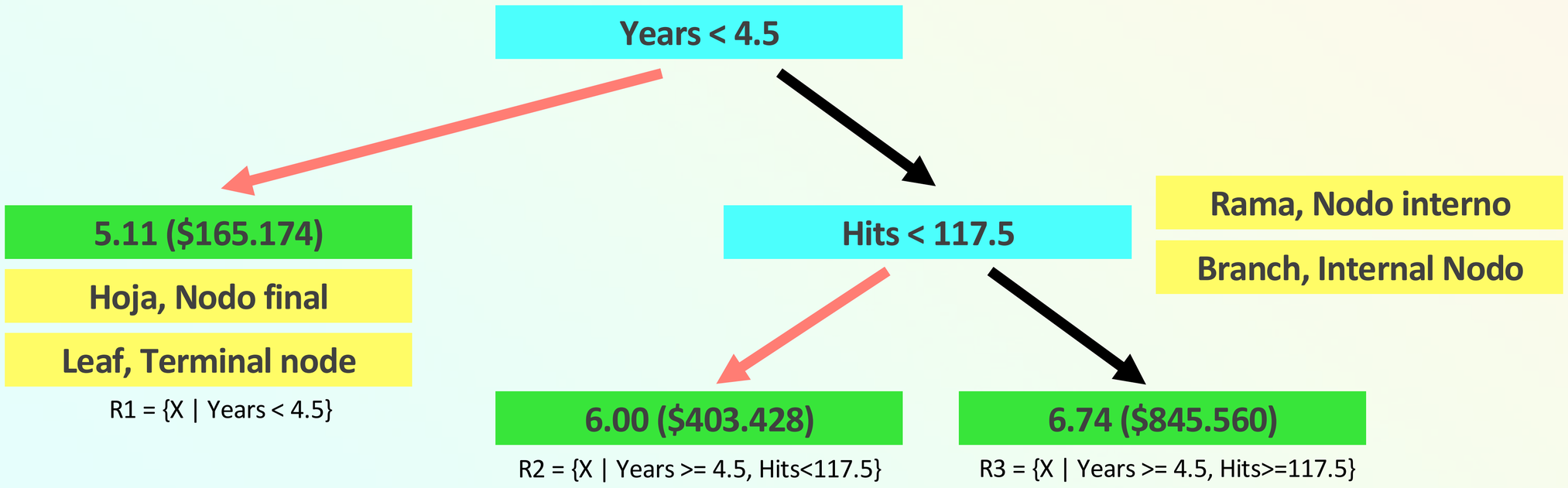
Los árboles de clasificación y regresión, conocidos como CART (Classification and Regression Trees), son una poderosa técnica de aprendizaje automático que se utiliza ampliamente para resolver problemas tanto de clasificación como de regresión.

Los árboles CART son modelos de decisión que emplean una estructura jerárquica en forma de árbol para realizar predicciones basadas en reglas lógicas sencillas y fáciles de interpretar.

Arboles de clasificación

Arboles de regresión

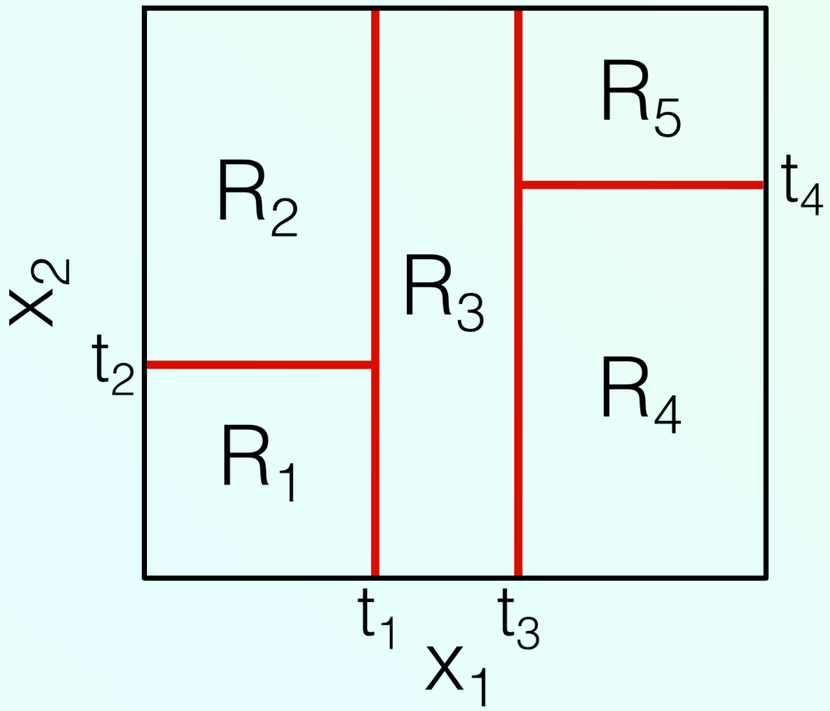
ARBOLES DE REGRESIÓN

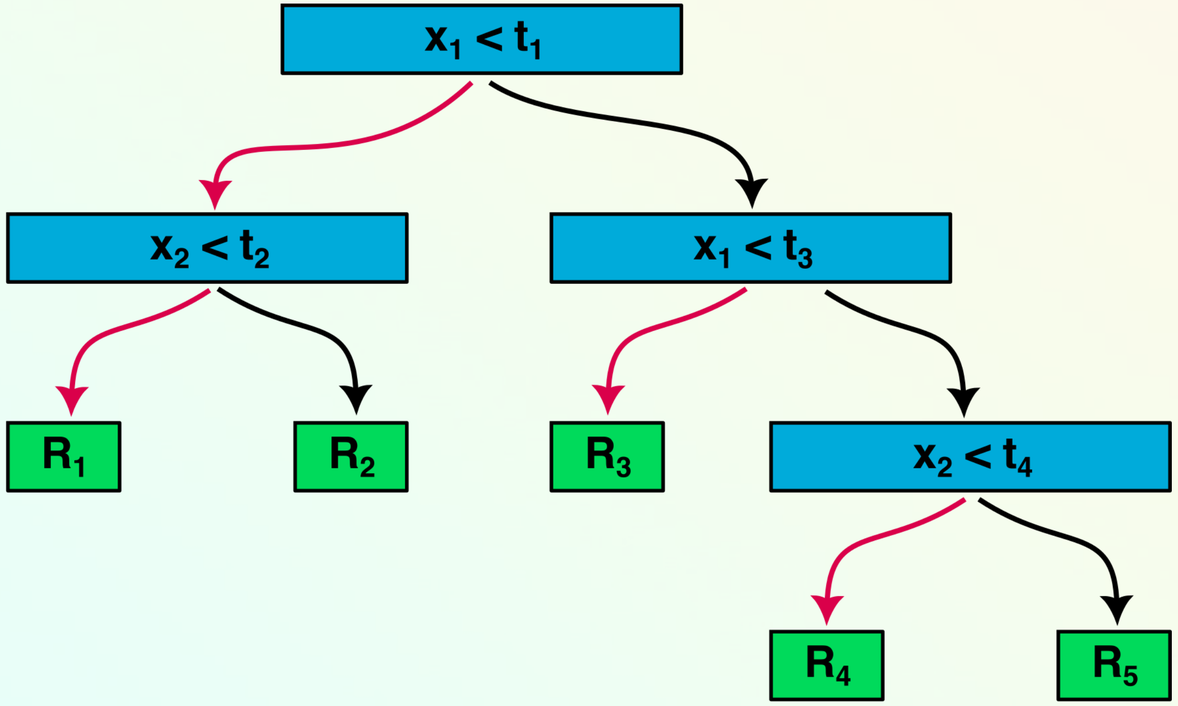


ARBOLES DE REGRESIÓN

Recursive binary splitting

$$RSS = \sum_{j=1}^{J} \sum_{i \in R_j} \left(y_i - \hat{y}_{R_j} \right)^2$$





ARBOLES DE REGRESIÓN

Poda de árboles

Introducimos un valor de penalización α a nuestra fórmula de RSS. Dado un valor de α, existe un sub-árbol $T \subset T_0$ tal que la expresión:

$$\sum_{m=1}^{L} \sum_{i \in R_j} \left(y_i - \hat{y}_{R_m}\right)^2 + \alpha L$$

...es mínima. Aquí, L indica el número de hojas del sub-árbol.

El parámetro α (α≥0) presenta un *trade-off* (compromiso) entre la complejidad del árbol y su capacidad de ajustarse a los datos (Regularización).

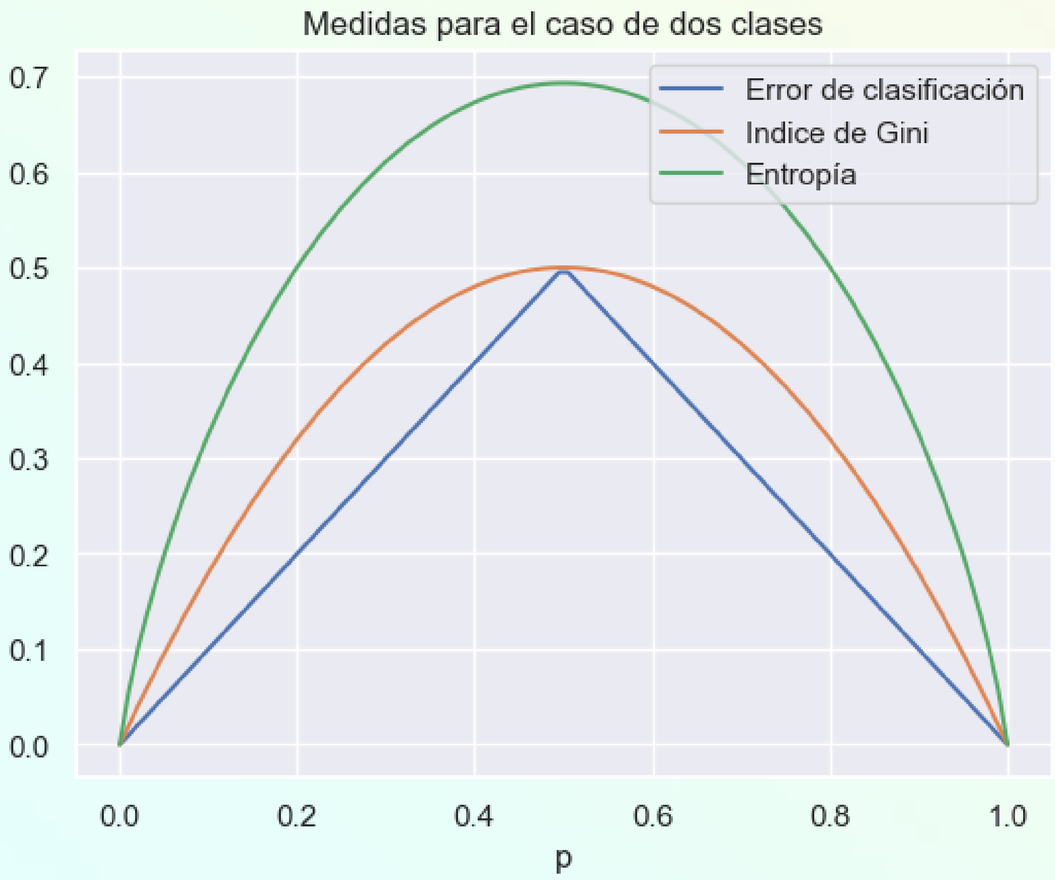
ARBOL DE CLASIFICACIÓN

Un árbol de clasificación es muy similar a uno de regresión, con la diferencia de que se utiliza para predecir una variable cualitativa.

La clase predicha se determina según la clase más frecuente (moda) entre las muestras que se encuentran en la hoja.

Al interpretar los resultados, a menudo no solo nos interesa la predicción de clase para un nodo terminal específico, sino también las proporciones de cada clase entre las observaciones de entrenamiento que caen en esa región.

ARBOL DE CLASIFICACIÓN



MÉTODOS DE ENSAMBLE

MÉTODOS DE ENSAMBLE

¿Qué son?

En 1906, el estadístico Sir Francis Galton visitó una feria rural en Inglaterra donde se celebraba un concurso para adivinar el peso de un buey en exhibición. Se realizaron 800 estimaciones y, aunque las respuestas individuales variaron enormemente, tanto la media como la mediana de todas ellas resultaron estar a menos del 1% del peso real del animal.

→ **Wisdom of the crowd (Sabiduría de la multitud)**

Si trasladamos esta idea a Machine Learning, observamos que al promediar la salida de un conjunto de predictores es muy probable obtener un mejor rendimiento que al considerar la salida de uno solo.

Dado que a un grupo de predictores se le conoce como **ensamble**, a esta técnica se le denomina **Métodos de Ensamble**.

MÉTODOS DE ENSAMBLE

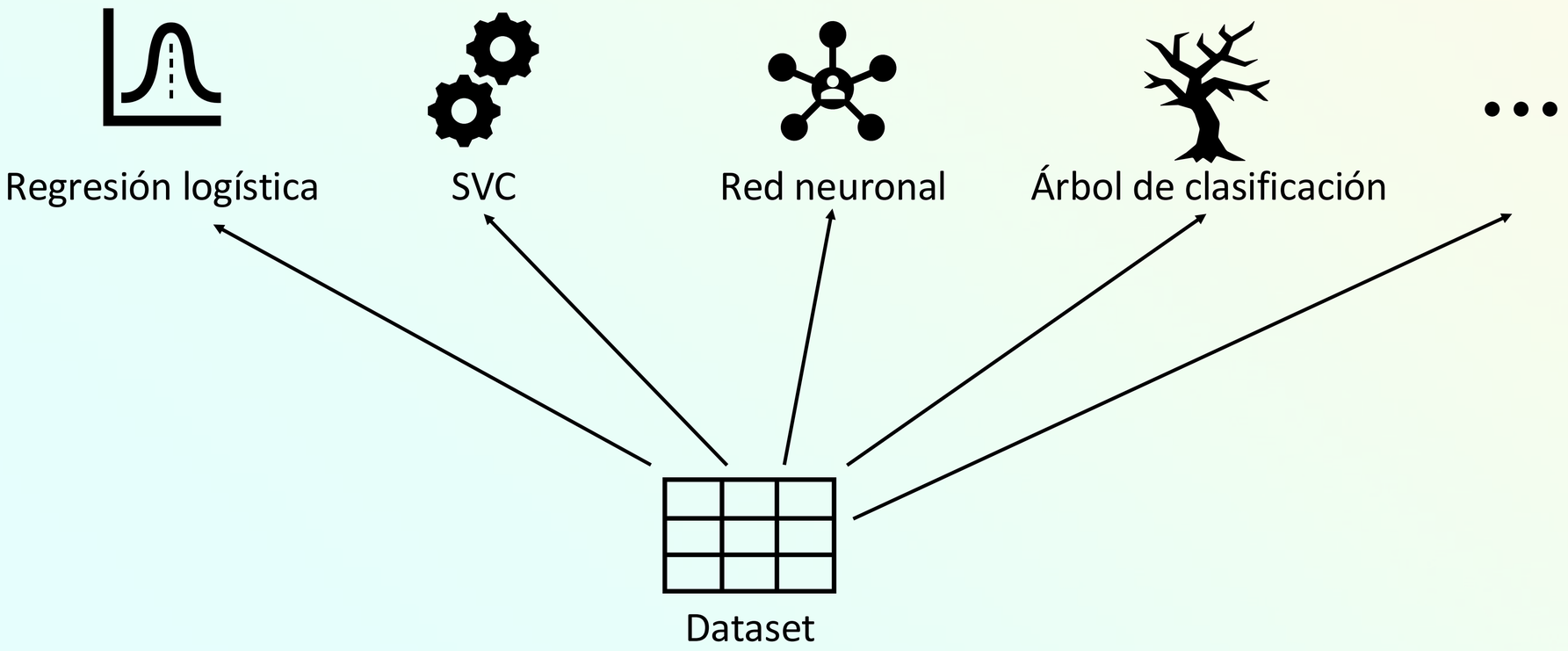
Por ejemplo, podríamos entrenar un grupo de árboles de decisión, utilizando para cada uno un **subconjunto diferente** de los datos de entrenamiento. Luego, generamos predicciones con cada árbol individual y determinamos la clase final mediante una **votación mayoritaria**.

¿Cuándo usarlos?

Es frecuente utilizar métodos de ensamble cuando las arquitecturas simples (conocidas como *weak learners*) no logran el rendimiento esperado. Para mejorar las métricas, combinamos estos predictores en un único modelo robusto.

MODELOS QUE VOTAN

MODELOS QUE VOTAN



MODELOS QUE VOTAN

Hard Voting: Estrategia basada en la moda. Se entrena un conjunto de modelos y la clase final predicha corresponde a la que recibe la mayoría de los votos por parte de los estimadores individuales.

Soft Voting: Si los clasificadores pueden estimar probabilidades de clase (p. ej., tienen el método *predict_proba*), la predicción se determina promediando las probabilidades de cada clase a través de todos los clasificadores y seleccionando la clase con el promedio más alto (*argmax*).

Voting Regressor: Para variables de respuesta continuas, la predicción del ensamble se obtiene calculando la media aritmética de las salidas individuales de los regresores.

MODELOS QUE VOTAN

Hard Voting: Estrategia basada en la moda. Se entrena un conjunto de modelos y la clase final predicha corresponde a la que recibe la mayoría de los votos por parte de los estimadores individuales.

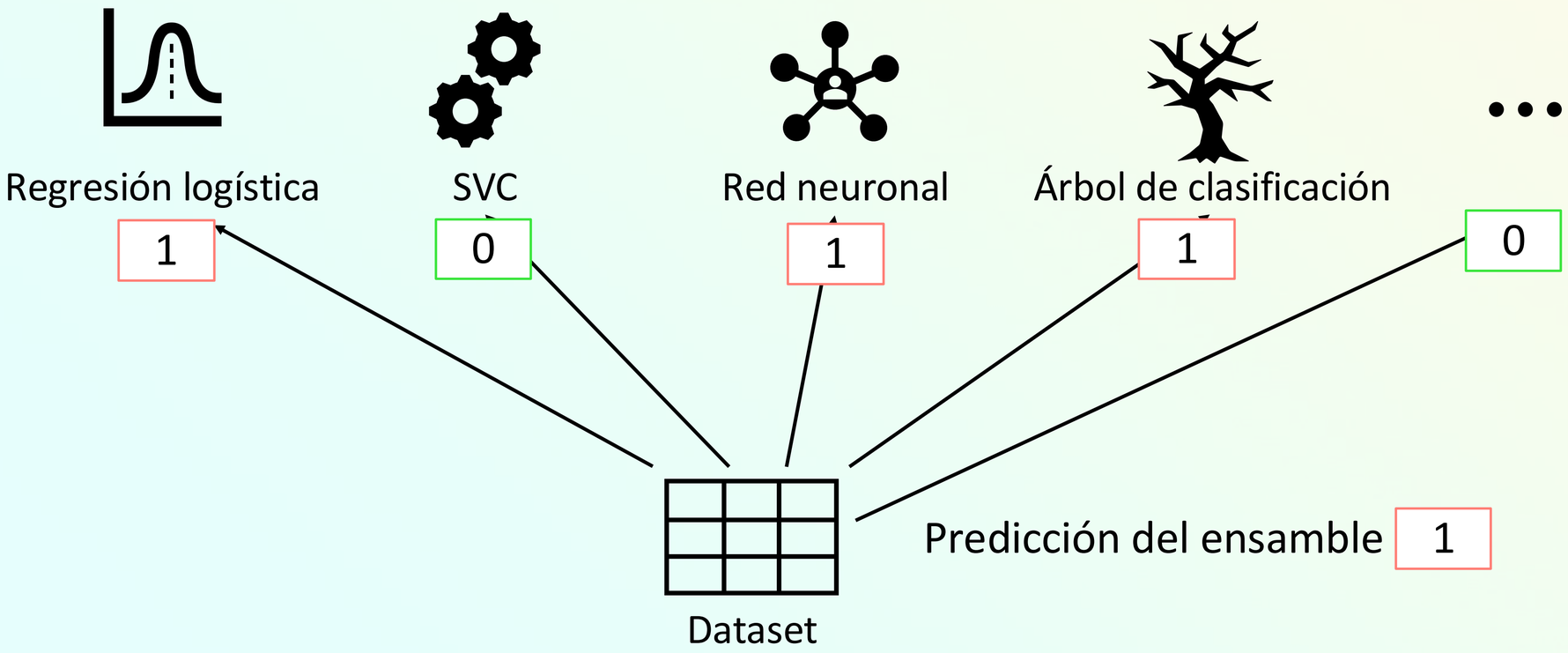
Soft Voting: Si los clasificadores pueden estimar probabilidades de clase (p. ej., tienen el método *predict_proba*), la predicción se determina promediando las proba todos los clasificadores y seleccionando la clase con el promedio más

Una ventaja de este método es que otorga mayor peso a los modelos que tienen más confianza en su decisión

Voting Regressor: Para variables de respuesta continuas, la predicción del ensamble se obtiene calculando la media aritmética de las salidas individuales de los regresores.

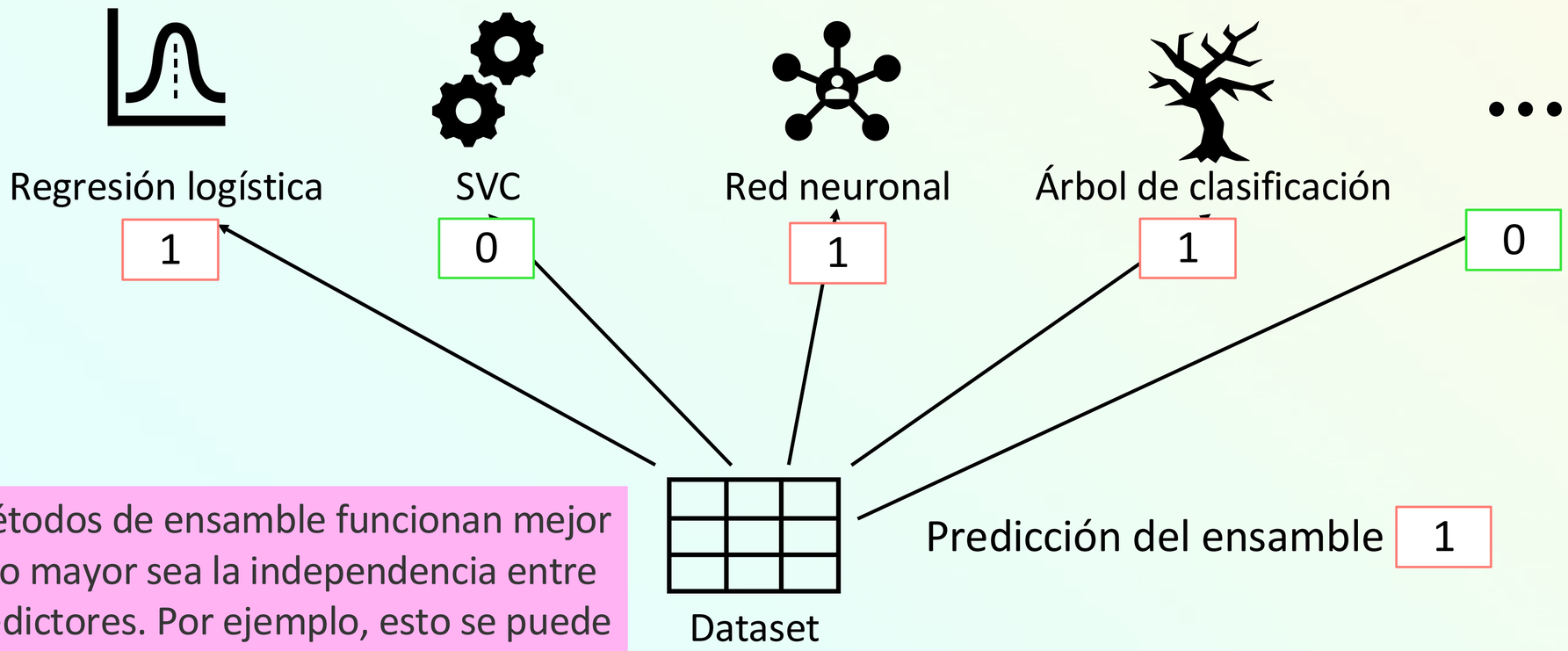
MODELOS QUE VOTAN

Hard voting classifiers



MODELOS QUE VOTAN

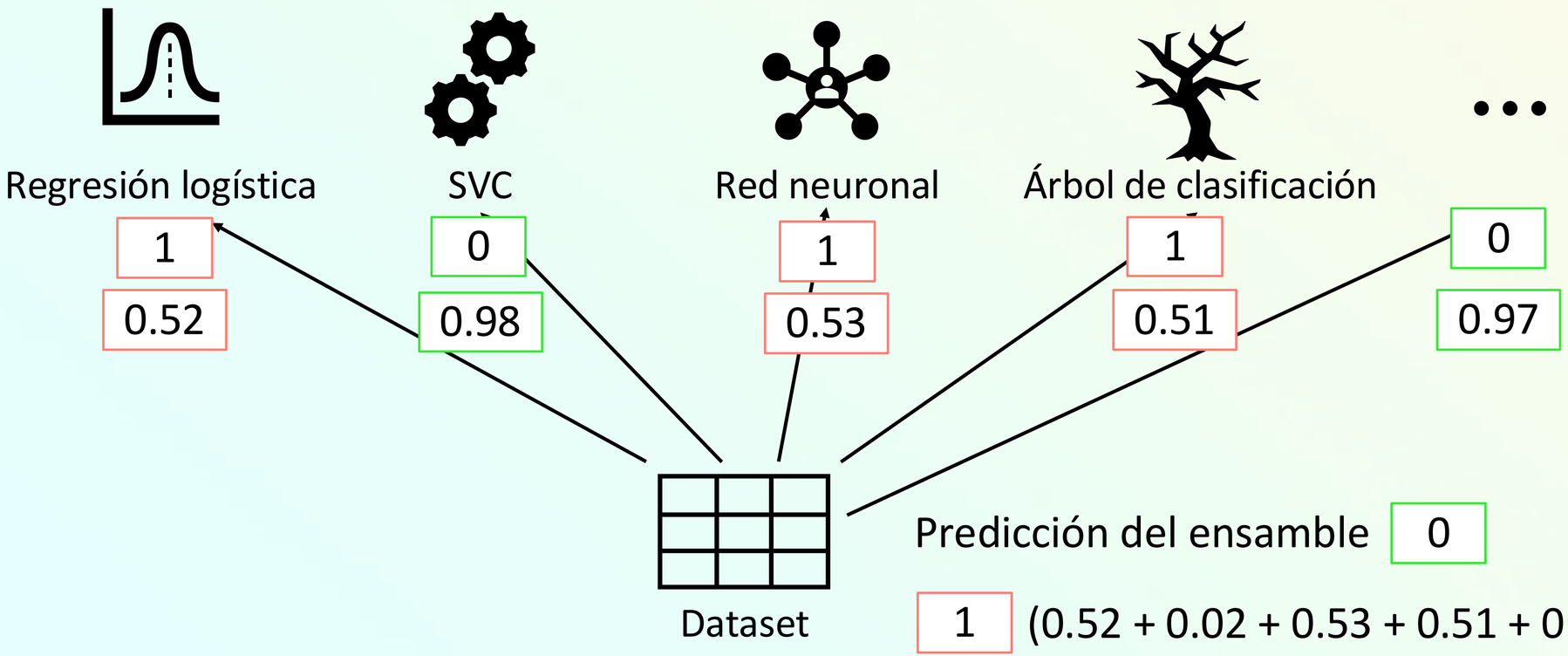
Hard voting classifiers



Los métodos de ensamble funcionan mejor cuanto mayor sea la independencia entre los predictores. Por ejemplo, esto se puede lograr utilizando arquitecturas distintas para cada modelo.

MODELOS QUE VOTAN

Soft voting classifiers



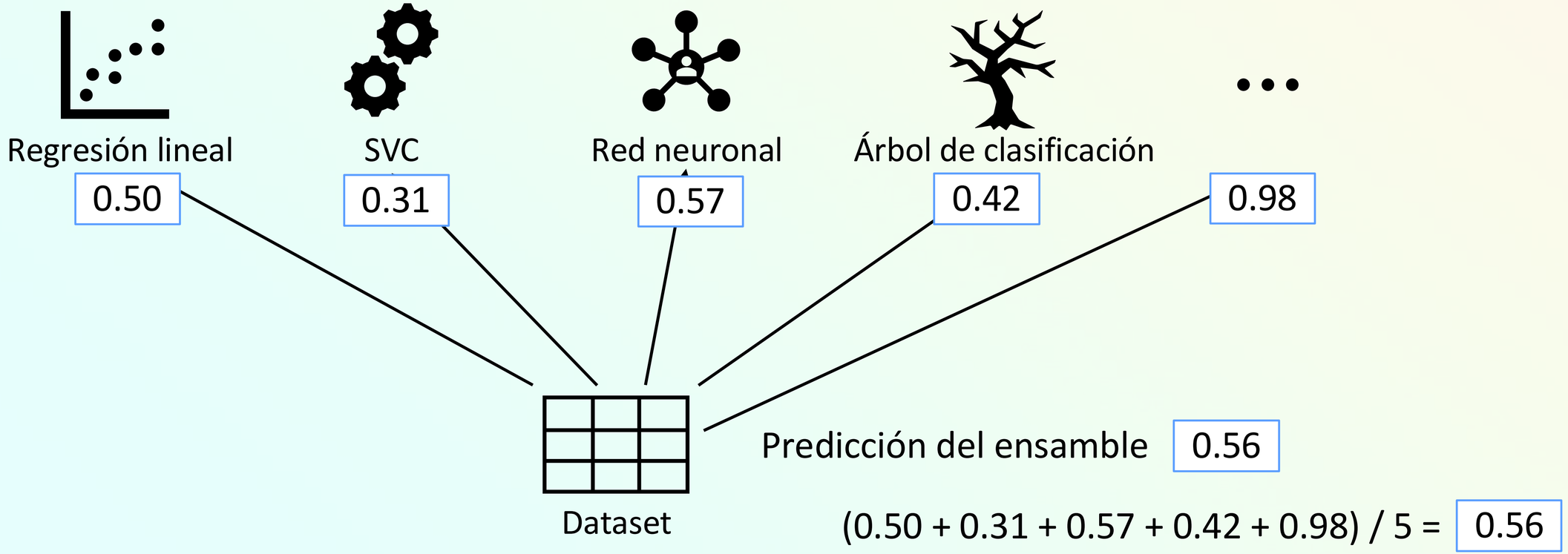
Predicción del ensamble 0

1 (0.52 + 0.02 + 0.53 + 0.51 + 0.03) / 5 = 0.312

0 (0.48 + 0.98 + 0.47 + 0.49 + 0.97) / 5 = 0.678

MODELOS QUE VOTAN

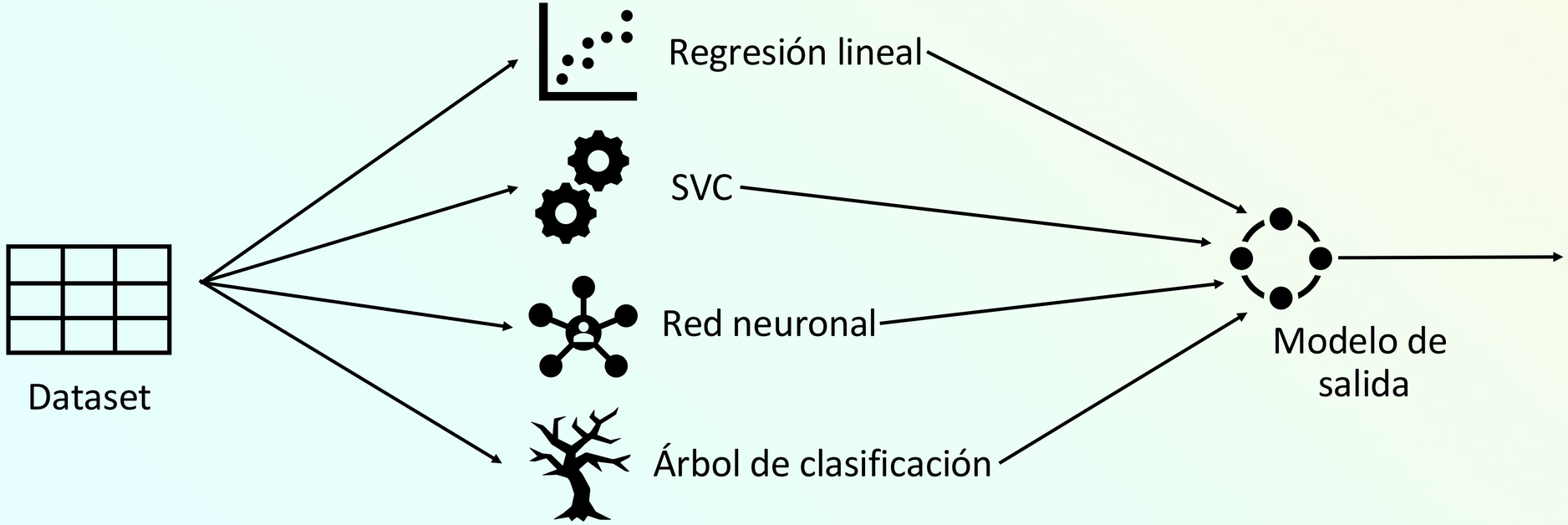
Voting regression



MODELOS QUE VOTAN

Stacking (Apilamiento)

¿Podemos mejorar la votación simple? Sí. Una estrategia más sofisticada es entrenar un meta-modelo (o modelo final) que aprende a combinar las salidas de los predictores bases para realizar la predicción definitiva.



VAMOS A PRÁCTICAR UN POCO…

BAGGING

BAGGING

¿Cómo logramos diversidad en nuestros predictores?

Una forma de obtener diversidad en el ensamble es utilizando arquitecturas diferentes para cada predictor.

Otra alternativa es usar la misma arquitectura, pero entrenándola con distintos subconjuntos del set de datos.

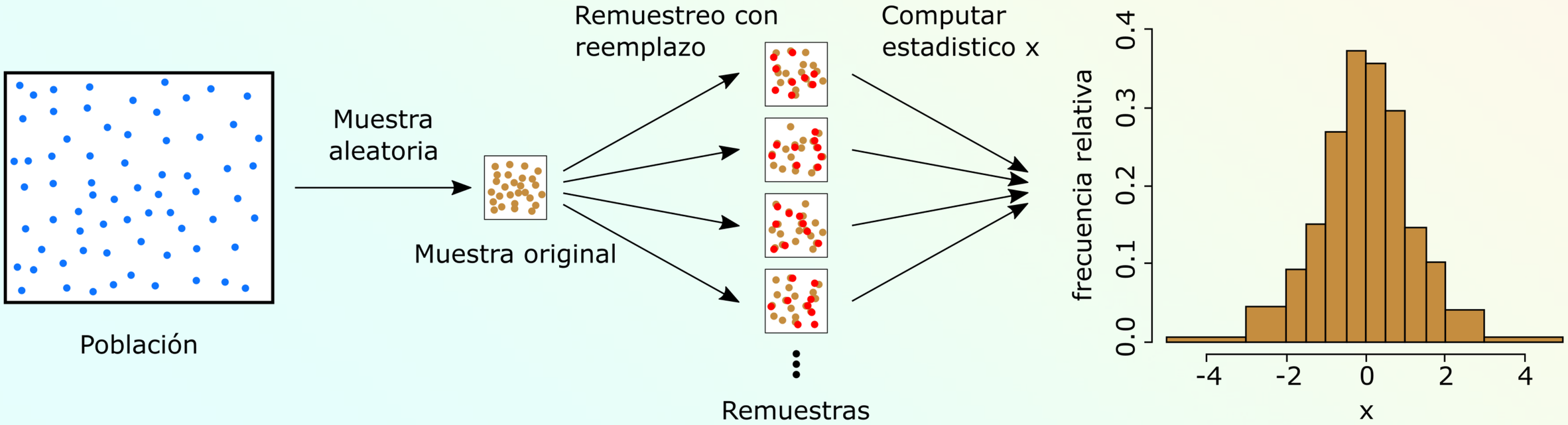
Dependiendo del tipo de muestreo, distinguimos dos métodos:

- **Bagging**: El muestreo se realiza **con reposición** (reemplazo).
- **Pasting** : El muestreo se realiza **sin reposición**.

Si bien en ambos casos una fila puede ser usada por distintos predictores, la clave es que solo en **Bagging** una misma observación puede aparecer repetida varias veces dentro del entrenamiento de un mismo predictor.

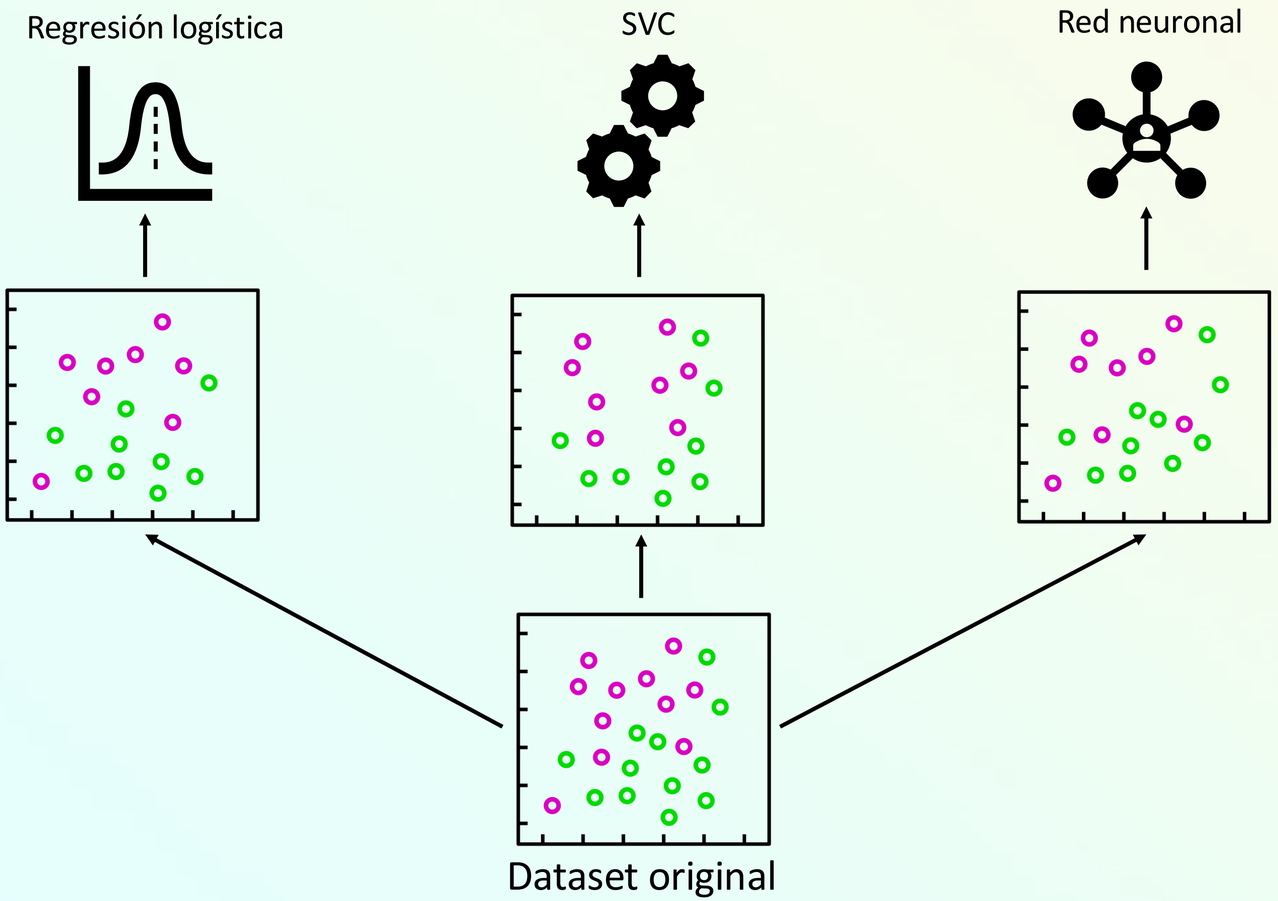
BAGGING

En estadística, la técnica de re-muestreo con reposición se denomina **Bootstrapping**



BAGGING

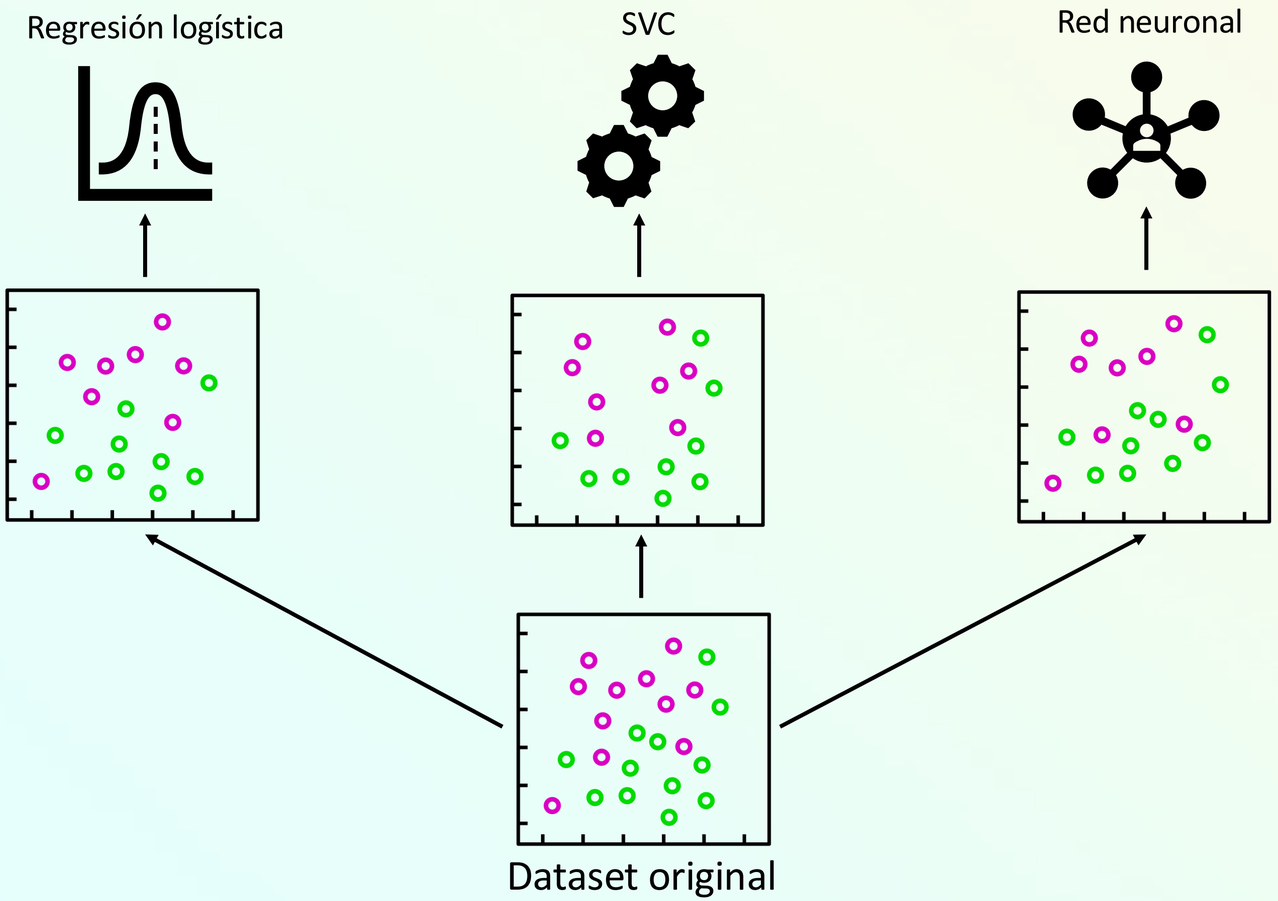
Para realizar **Bagging (bootstrap aggregating)**, utilizamos modelos que tengan mucha varianza y poco sesgo.



BAGGING

Para realizar **Bagging (bootstrap aggregating)**, utilizamos modelos que tengan mucha varianza y poco sesgo.

Cada uno de los modelos del ensamble puede ser entrenado en paralelo, lo que permite escalar la solución.



Re-muestreo con reemplazo

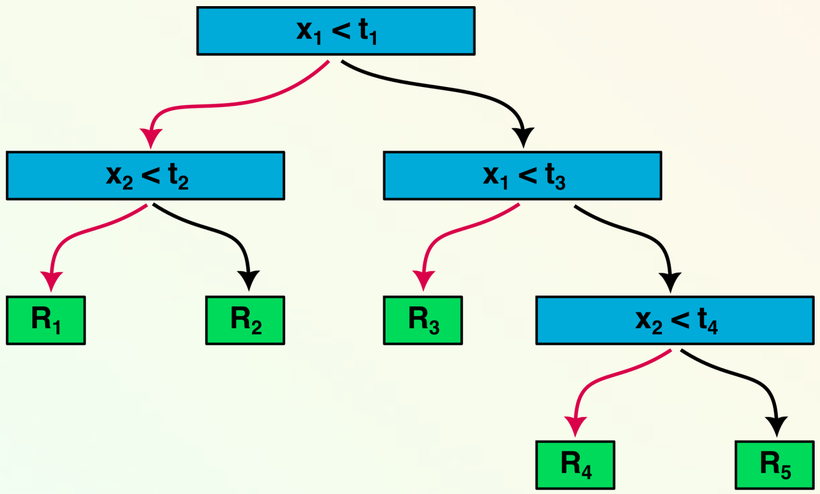
BAGGING

Lo más común es utilizar un solo tipo de modelo base, particularmente el **árbol de decisión**.

- Si el problema es de clasificación, los árboles deciden por votación.
- Si es de regresión, se promedian las predicciones.

Dado que cada árbol se entrena con una muestra distinta, el resultado será diferente al de los demás. Generalmente, se usan **árboles profundos** (es decir, con alta **varianza** y poco **sesgo**). Al promediar sus salidas, logramos reducir esa varianza.

Bagging mejora sustancialmente los resultados cuando se escala a cientos o miles de árboles.



BOSQUES ALEATORIOS

BOSQUES ALEATORIOS

Los **bosques aleatorios** (Random Forests) son una mejora respecto al uso exclusivo de **Bagging**.

El funcionamiento es similar, pero agregan un paso crucial: en cada división se considera solo **una selección aleatoria de atributos**. Por regla general, la cantidad de atributos a utilizar (m) es aproximadamente la raíz cuadrada del total de atributos disponibles (p).

Ejemplo: Si tenemos p=13 atributos, usaremos m≈4 (ya que 13≈3.6). En cada nodo del árbol, el algoritmo elegirá una combinación diferente de 4 atributos al azar para decidir la mejor división.

BOSQUES ALEATORIOS

El razonamiento:

Supongamos que existe un atributo que es un **predictor muy fuerte**, acompañado de otros moderadamente fuertes.

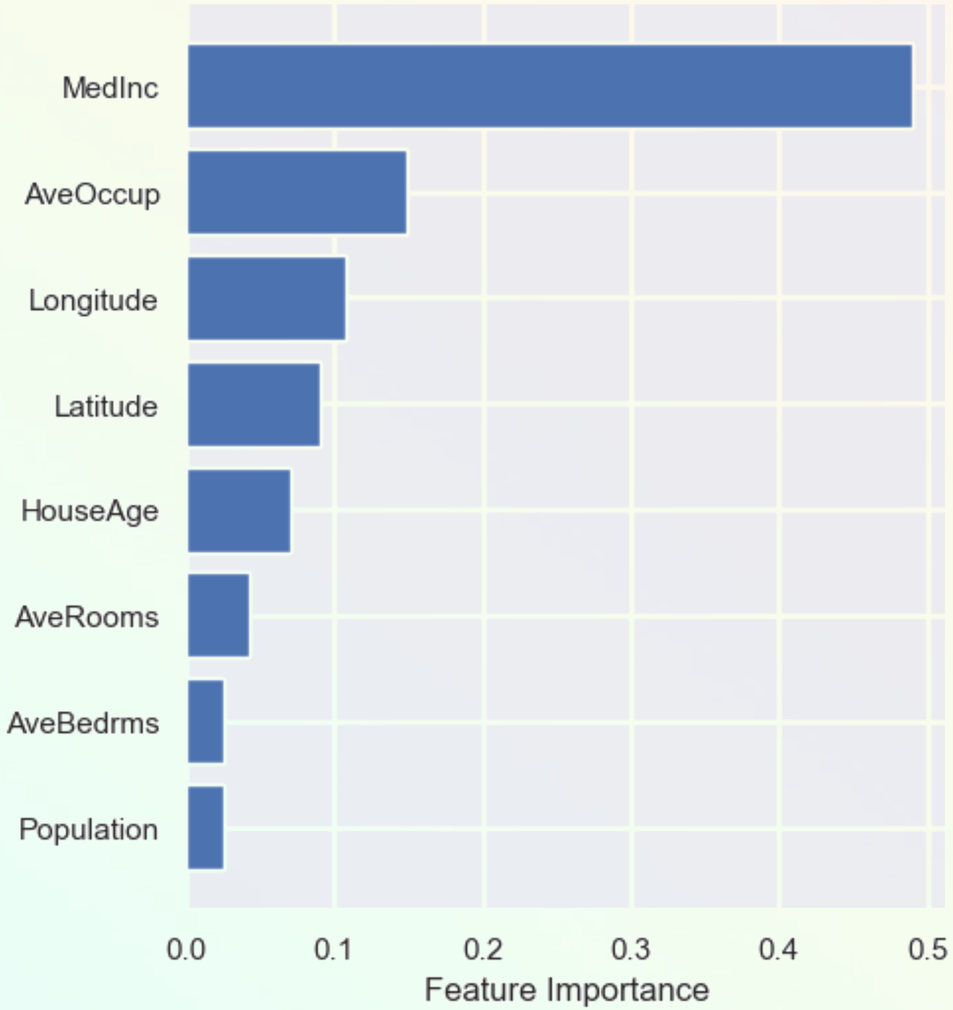
Si aplicamos **Bagging**, la gran mayoría de los árboles tenderán a utilizar ese atributo fuerte en la bifurcación inicial (nodo raíz). Por consiguiente, los árboles resultantes serán muy similares entre sí, generando una alta correlación y **limitando la capacidad del ensamble para reducir la varianza**.

Random Forests soluciona esto al restringir los atributos disponibles en cada nodo. En promedio, una fracción de **(p-m)/p** de las divisiones no tendrá en cuenta ese atributo fuerte, lo que obliga a los árboles a explorar otras variables y logra descorrelacionar las predicciones.

BOSQUES ALEATORIOS

El poder de los Bosques Aleatorios se hace evidente al crear modelos para datos con **muchos atributos**.

Estos modelos tienen la capacidad de determinar automáticamente qué variables son importantes y descubrir relaciones complejas entre ellas (interacciones), que a menudo pasan desapercibidas para otros algoritmos.



BOSQUES ALEATORIOS

Existen dos enfoques principales:

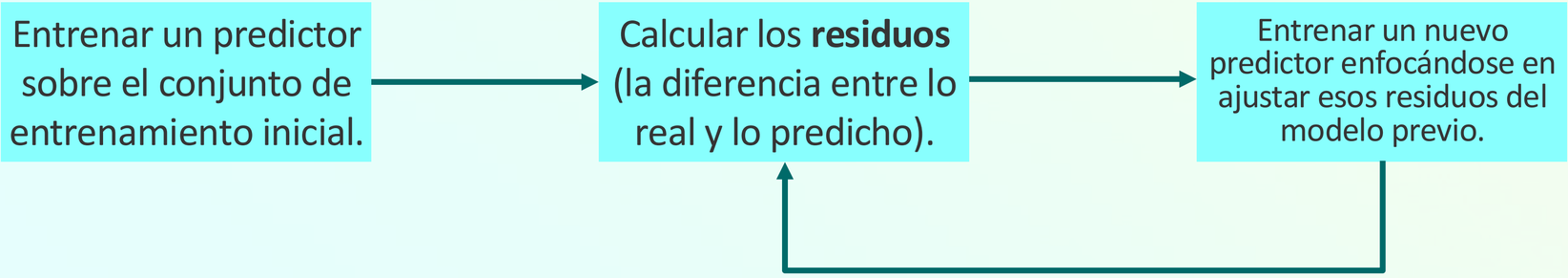
- **Por disminución de la precisión (Permutación):** Se mide cuánto cae la precisión del modelo si los valores de un atributo se permutan aleatoriamente. Al mezclar los valores de una columna, rompemos su relación con la variable objetivo, eliminando su poder predictivo. Si el error aumenta mucho, el atributo era importante.
- **Por disminución de la impureza (Impureza media):** Se calcula la disminución media en el criterio de impureza (Índice Gini, Entropía) acumulada por todos los nodos divididos por ese atributo. Básicamente, mide cuánto mejora la pureza (homogeneidad) de los nodos gracias a esa variable.

BOOSTING

BOOSTING

Este tipo de arquitecturas son ensambles que, al igual que los vistos anteriormente, combinan varios predictores para lograr una mejor predicción. La diferencia principal radica en que, en **Boosting**, el entrenamiento de los predictores ocurre de forma secuencial.

Cada nuevo predictor que se agrega al ensamble intenta corregir los errores de su predecesor.



BOOSTING

Existen varios algoritmos de Boosting con distintos niveles de complejidad, pero el más sencillo de entender es **Adaboost**. Su proceso de entrenamiento simplificado es el siguiente:

1. **Inicialización**: Se comienza asignando **pesos iguales** a todas las muestras del conjunto de entrenamiento.

2. **Entrenamiento:** Se entrena un modelo base (*weak learner*) utilizando los datos ponderados.

3. **Evaluación:** Se calcula el error del modelo. Las observaciones mal clasificadas adquieren mayor importancia para el siguiente paso.

4. **Peso del modelo (α):** Se asigna un peso al modelo en función de su precisión: los modelos más acertados tendrán mayor influencia en la votación final.

5. **Actualización de pesos:** Se incrementan los pesos de las instancias mal clasificadas para que el próximo modelo se vea "forzado" a enfocarse en ellas.

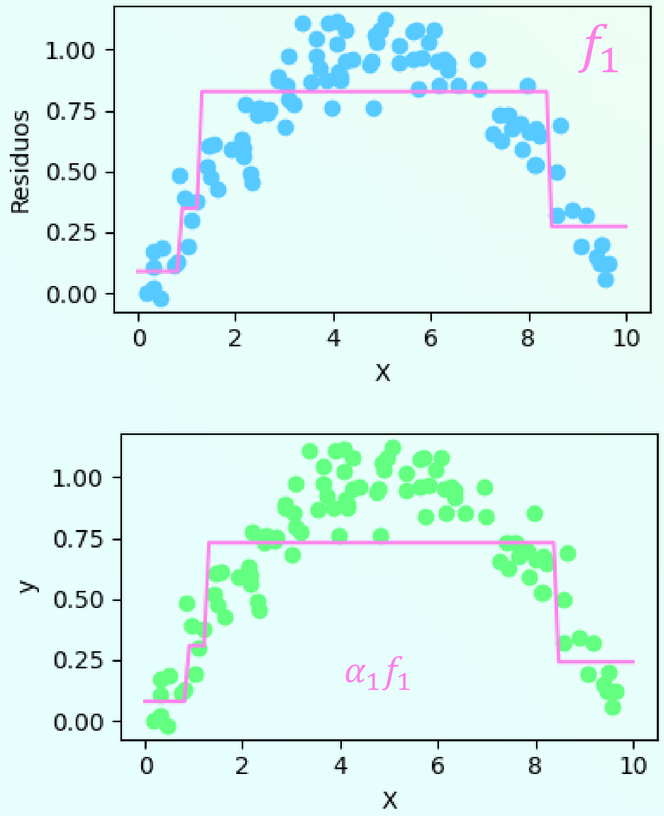
6. **Iteración:** Se repiten los pasos 2 a 5 varias veces (generalmente de 50 a 100 iteraciones).

Predicción Final: Finalmente, se combinan las predicciones de todos los modelos débiles, ponderadas por sus respectivos pesos (α), mediante la fórmula:

$$\hat{F}(X) = \sum_{m=1}^{M} \alpha_m \hat{f}_m(X)$$

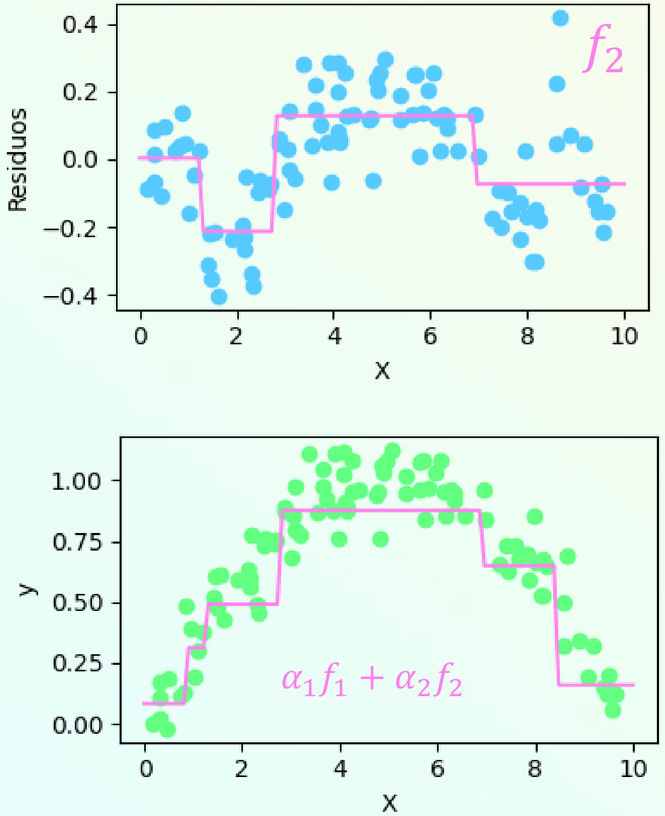
BOOSTING

Iteración 1



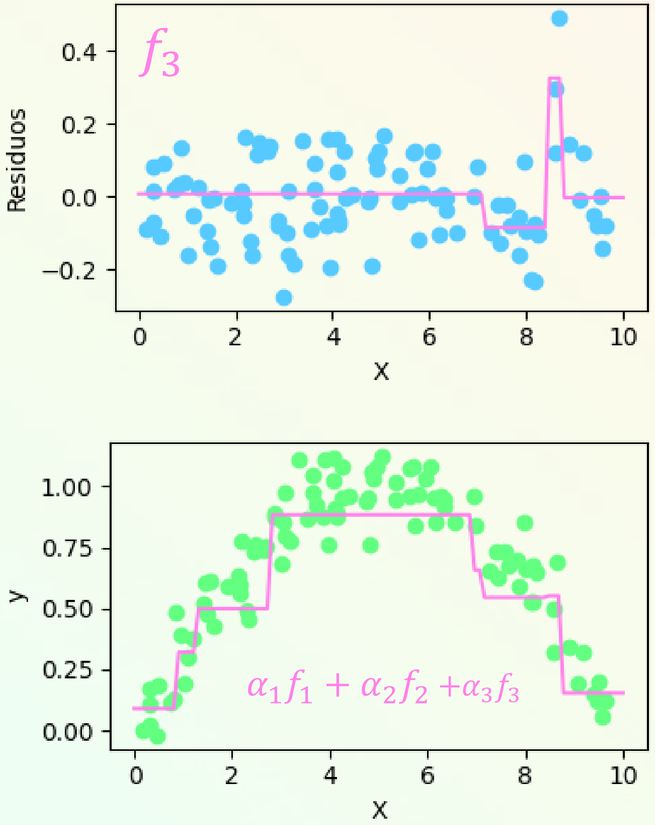
Salida de ensamble: $F = \alpha_1 f_1$

Iteración 2



Salida de ensamble: $F = \alpha_1 f_1 + \alpha_2 f_2$

Iteración 3



Salida de ensamble: $F = \alpha_1 f_1 + \alpha_2 f_2 + \alpha_3 f_3$

BOOSTING

Entrenamiento Secuencial

La principal diferencia en las arquitecturas de **Boosting** es que el entrenamiento de los predictores se realiza de forma secuencial.

Lo más común es utilizar árboles de decisión con pocos nodos (poco profundos). Este es un proceso de aprendizaje lento, ya que cada paso depende de los árboles anteriores. Sin embargo, estadísticamente, un proceso de aprendizaje lento suele conducir a mejores resultados (o un mejor modelo final).

BOOSTING VS. BAGGING

	Bagging	Boosting
Método de construcción del conjunto de modelos	Múltiples modelos de aprendizaje automático independientes entrenados en subconjuntos aleatorios de datos de entrenamiento	Construye modelos secuencialmente, centrándose en corregir los errores del modelo anterior
Tipo de modelos	Adecuado para cualquier modelo de aprendizaje automático	Se enfoca en modelos débiles y mejora su precisión al combinar modelos débiles
Proceso de predicción	Promedio de las predicciones de todos los modelos del conjunto	Asigna diferentes pesos a cada modelo y los combina para hacer una predicción final
Sensibilidad al ruido	Menos sensible al ruido y a los valores atípicos	Puede ser más sensible al ruido y a los valores atípicos
Tiempo de entrenamiento	Puede ser más rápido ya que los modelos se construyen de forma independiente	Puede requerir más tiempo de entrenamiento ya que cada modelo se entrena secuencialmente
Ventajas	Reduce la varianza, mejora la precisión y puede manejar grandes conjuntos de datos	Mejora la precisión y el rendimiento del modelo mediante la corrección de errores y la creación de modelos más precisos
Desventajas	Puede aumentar el sesgo y no mejorar la precisión si los modelos del conjunto son similares	Puede ser más propenso al sobreajuste y al ruido si se usa un modelo débil o ruidoso

XGBOOST

XGBOOST

Una de las librerías más utilizada en Boosting es **XGBoost**, una implementación de Gradient Boosting estocástico desarrollada por Tianqi Chen y Carlos Guestrin en la Universidad de Washington.

Mientras que en AdaBoost se usa una función exponencial para ponderar las observaciones, en XGBoost se utiliza el gradiente de la función de costo. Con este gradiente se ajustan los residuos y se ponderan los nuevos modelos (weak learners).

XGBOOST

El modelo XGBoost cuenta con una gran cantidad de hiperparámetros, pero dos de los más importantes son:

- **subsample**: Controla la fracción de observaciones que se muestrean al azar en cada iteración. Al usar una submuestra, el algoritmo incorpora aleatoriedad (similar a un Random Forest), aunque aquí el muestreo se realiza típicamente sin reemplazo (**Pasting**).

- **eta**: Es un factor de contracción aplicado al peso α_m de cada nuevo árbol que se va agregando. Su función es escalar (reducir) la contribución de cada árbol nuevo al modelo total. Este parámetro es fundamental para controlar la velocidad de aprendizaje y evitar el sobreajuste (overfitting).

XGBOOST

En Python, **XGBoost** ofrece dos interfaces distintas:

- **Nativa**: Más cercana al paradigma funcional y similar a la implementación en R.
- **Scikit-Learn API**: Diseñada para imitar la estructura de Scikit-Learn. Para mantener la consistencia con esta librería, algunos parámetros cambian de nombre (por ejemplo, *eta* pasa a llamarse *learning_rate*)

Además, dada su tendencia al sobreajuste (overfitting), **XGBoost** incorpora potentes mecanismos de **regularización**. Dispone de dos parámetros principales para controlar la complejidad:

- *reg_alpha:* Corresponde a la regularización L1 (tipo Lasso).
- *reg_lambda:* Corresponde a la regularización L2 (tipo Ridge)

Incrementar estos valores penaliza la complejidad del modelo, lo que ayuda a reducir el tamaño de los pesos en las hojas y favorece árboles más conservadores.

Nota: Otra implementación muy popular de este tipo de algoritmos es LightGBM.

VAMOS A PRÁCTICAR UN POCO…